

Project 01: Smart Indoor Environment Monitoring and Automated Climate Control

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Build an end-to-end indoor IoT system that monitors environmental quality and automatically controls ventilation and/or lighting. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class MCU; temperature/humidity plus at least one of CO₂, VOC/air-quality, or illuminance.
- At least one real low-voltage actuator (fan, lamp, ventilation model) with an appropriate relay/MOSFET/driver.
- Document sensor calibration or sanity checking; raw ADC values alone are not accepted as final measurements.

Mandatory IoT, Backend, and Application Requirements

- MQTT telemetry with documented topic hierarchy, device ID, timestamps, justified QoS, and online/offline status using LWT or equivalent.
- Backend must ingest telemetry, store historical data, and expose REST API(s) for status/history/configuration.
- Real-time dashboard: current values, historical plots, data freshness, actuator state, alarms, and Auto/Manual mode.
- Automatic control must use hysteresis, minimum switching time, state-machine logic, or another anti-chattering method; a single unprotected threshold is insufficient.
- Define safe local behavior and automatic reconnection when network/backend communication is lost.

Required Advanced Component

Use a multi-variable comfort/air-quality rule involving at least two measurements, or implement occupancy-aware control. Justify all thresholds and control logic. **Experimental Evaluation — Mandatory**

- Measure telemetry latency and command-to-actuator response time.
- Test at least normal operation, environmental threshold violation, and network interruption/recovery.
- Quantify sensor/control performance using error, response time, switching/chattering count, packet success, or time within desired environmental range.

Live Demonstration Requirement

Live sensor changes must appear on the dashboard; automatic control must physically activate an actuator; manual override and network recovery must be demonstrated. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 02: Smart Agriculture Monitoring and Automated Irrigation

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Develop a closed-loop irrigation system that observes soil/environmental conditions, makes irrigation decisions, controls a pump/valve, and provides remote monitoring and historical analysis. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class node; soil-moisture sensor plus temperature/humidity or illuminance; water-level sensing is strongly recommended.
- Low-voltage pump/valve through a correctly rated driver; provide flyback protection for inductive loads where required.
- Calibrate soil moisture using at least dry and wet reference conditions and document the conversion from raw reading to engineering scale.

Mandatory IoT, Backend, and Application Requirements

- Design MQTT telemetry/command topics, QoS, timestamps, and availability state.
- Store time-series measurements, irrigation events, pump runtime, and alarms; expose REST history/configuration API(s).
- Dashboard: soil/environment trends, water/pump state, Auto/Manual mode, thresholds, alarms, and irrigation history.
- Control must include hysteresis and minimum ON/OFF time or a finite-state controller; prevent pumping from an empty source if level sensing is provided.
- Local automatic irrigation must continue according to a documented safe policy during connectivity loss.

Required Advanced Component

Use at least two variables in the irrigation decision (e.g., soil moisture plus temperature, light, or time-of-day), or implement multiple independently controlled zones. **Experimental Evaluation — Mandatory**

- Characterize moisture-sensor response/repeatability.
- Compare the proposed strategy with a fixed-threshold or fixed-schedule baseline using water use, pump runtime, or time within target moisture range.
- Measure communication/control latency and verify network-loss/reconnection behavior.

Live Demonstration Requirement

Physically alter moisture or controlled sensor conditions, trigger automatic irrigation, show remote manual control, and demonstrate at least one safe fault response. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 03: Smart Energy Monitoring and Intelligent Load Management

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Build an IoT energy-management prototype that measures consumption, stores and visualizes energy data, detects excessive demand, and automatically manages controllable loads. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class node and suitable isolated/low-voltage current/energy measurement. Student-built exposed mains-voltage measurement circuits are prohibited.
- At least two controllable low-voltage loads, or one controllable load plus an independently measured reference load.
- Use appropriately rated relay/MOSFET/driver hardware and document electrical ratings/protection.

Mandatory IoT, Backend, and Application Requirements

- Acquire and publish current plus power/energy-related telemetry with documented computation, units, and sampling/update rate.
- MQTT with structured topics/QoS and availability; time-series database and REST API for history/configuration.
- Dashboard: instantaneous demand, accumulated energy, historical trend, estimated consumption/cost, load states, and alarms.
- Implement priority/rule-based load management; e.g., shed a low-priority load above a configurable demand limit and restore only after stable safe conditions.
- Remote switching must distinguish requested command state from acknowledged/confirmed device state.

Required Advanced Component

Add peak-demand detection, daily energy budgeting, or an explainable load-scheduling algorithm. **Experimental Evaluation — Mandatory**

- Compare measurements against a reference instrument/known load and report error.
- Measure event-to-dashboard and command-to-confirmation latency.
- Compare unmanaged versus managed operation using peak power, energy, or threshold-violation duration; include disconnect/recovery testing.

Live Demonstration Requirement

Demonstrate live power changes, historical recording, threshold alarm, automatic load shedding/restoration, manual override, and safe network-loss behavior. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 04: Edge-IoT Predictive Maintenance for Rotating Machinery

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Design a condition-monitoring system that senses a safe rotating-machine rig, extracts meaningful edge features, detects abnormal operation, and reports machine health through an IoT backend. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class MCU or MCU plus edge gateway; accelerometer is mandatory; add temperature, current, or RPM sensing.
- Justify vibration sampling frequency from the bandwidth of interest and discuss anti-alias requirements.
- Use a safe low-voltage motor/fan rig; deliberate destructive faults are prohibited.

Mandatory IoT, Backend, and Application Requirements

- Perform local feature extraction such as RMS, peak-to-peak, crest factor, dominant frequency, band energy, or FFT-derived features; raw streaming alone is insufficient.
- Publish health features/events through MQTT; store historical condition data; expose REST API(s); provide real-time dashboard and alarms.
- Implement an explainable condition classifier using thresholds, statistical methods, or lightweight ML/TinyML. Deep learning is optional.
- Local sensing/detection must continue during network failure; device availability and data freshness must be visible.

Required Advanced Component

Implement TinyML or a multi-feature anomaly score and compare it with a simpler threshold baseline. If ML is used, explain dataset creation, labels, train/test separation, and deployment constraints. **Experimental Evaluation — Mandatory**

- Collect and document a labeled dataset for at least two operating conditions.
- Report suitable detection metrics: accuracy, precision/recall, confusion matrix, false-alarm and/or missed-detection rate.
- Measure edge-processing latency/resource use, communication latency, and network-interruption behavior.

Live Demonstration Requirement

Run normal and abnormal conditions live, show edge features and health state, trigger an alarm, and demonstrate continued local detection without the network. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 05: Real-Time Smart Parking Management and Guidance

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Implement a multi-slot IoT parking prototype that detects occupancy, maintains consistent global state, visualizes changes in real time, and guides users to an available slot. **Mandatory Hardware and Physical-System Requirements**

- At least four physical parking slots using ultrasonic, IR, magnetic, or justified alternative sensing; use at least two ESP32 nodes unless a distributed alternative is justified.
- Optional entrance/exit counter and servo barrier may be added with appropriate driver/power design.
- Every slot/node must have a stable logical identity independent of current sensor values.

Mandatory IoT, Backend, and Application Requirements

- Design hierarchical MQTT topics for zone/slot status, commands, and availability; justify QoS and retained messages.
- Backend must maintain current occupancy and historical utilization; expose REST API(s) and a real-time dashboard.
- Dashboard: parking map, free/occupied count, utilization, stale/offline slots, and history.
- Implement slot recommendation based on a documented cost such as distance, zone priority, or accessibility, including tie-breaking.
- Handle sensor debouncing, false transitions, node disconnection/reconnection, and duplicate/out-of-order updates where relevant.

Required Advanced Component

Add reservation with timeout/conflict handling, or entrance/exit consistency checking against slot occupancy. **Experimental Evaluation — Mandatory**

- Measure occupancy-detection accuracy over repeated trials.
- Measure update latency for single and near-simultaneous slot changes.
- Test node failure/recovery and evaluate global-state consistency and recommendation correctness.

Live Demonstration Requirement

At least four slots must update live; demonstrate recommendation, near-simultaneous changes, and clear detection of an offline node. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 06: Smart Building Access Control and Intrusion Detection

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Develop an IoT physical-security prototype combining authenticated access, door-state monitoring, intrusion detection, audit logging, alarms, and secure remote supervision. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class controller; RFID/NFC or keypad; door/reed sensor; PIR or equivalent intrusion sensor; buzzer/alarm; servo or safe low-voltage lock model.
- Define local access behavior when cloud/backend connectivity is unavailable.
- Actuator/alarm interfaces must be electrically appropriate and fail to a documented state.

Mandatory IoT, Backend, and Application Requirements

- Implement credential verification without transmitting plaintext secret credentials in telemetry.
- Use authenticated MQTT; separate telemetry, alarm, and command authorization where broker capabilities permit.
- Backend must record access attempts, door events, alarms, availability, and operator actions; provide REST queries and real-time dashboard.
- Implement a state machine distinguishing granted, denied, authorized opening, forced opening, alarm, and recovery.
- Use LWT/equivalent availability detection; remote reset/unlock requires authorization and acknowledged state.

Required Advanced Component

Use TLS and token-based dashboard/API authorization, or implement role-based permissions for administrator versus viewer. **Experimental Evaluation — Mandatory**

- Execute a test matrix: valid credential, invalid credential, forced opening, repeated failed attempts, network outage, and reconnection.
- Report event-detection latency, false-alarm behavior, audit-log completeness, and recovery behavior.
- Provide a concise threat model with at least five realistic attacks/misuse cases and mitigations.

Live Demonstration Requirement

Demonstrate complete valid access, denied access, forced-door alarm, audit logging, remote supervision, and network-loss behavior. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 07: Cold-Chain Monitoring and Intelligent Storage Management

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Build a cold-chain prototype that monitors environmental compliance over time, detects excursions, preserves traceable history, and distinguishes short disturbances from sustained violations. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class node; temperature and humidity sensing; door/open-close sensing is mandatory; optional second temperature/light sensor is encouraged.
- Justify sensor placement and compare/calibrate against a reference thermometer.
- Optional fan/cooling simulator may be controlled using safe low-voltage hardware.

Mandatory IoT, Backend, and Application Requirements

- Publish timestamped telemetry/door events through MQTT with availability; persist compliance-relevant history.
- Implement accumulated degree-time, excursion duration, or another justified duration-aware quality metric; instantaneous threshold alone is insufficient.
- Dashboard: current conditions, history, door events, exposure status, data freshness, alarms, and connectivity.
- REST API must support historical/compliance queries; distinguish warning from sustained/critical excursion.
- Use local buffering or local exposure calculation so network outage does not erase compliance information.

Required Advanced Component

Implement multi-level excursion severity jointly based on temperature deviation and duration, or compare two sensor locations for thermal non-uniformity. **Experimental Evaluation — Mandatory**

- Perform normal-storage, door-open, warming/cooling-loss, and network-outage experiments.
- Report sensor error, alarm delay, exposure calculation, data-loss/recovery performance, and false alarms.
- Experimentally demonstrate the added value of duration-aware exposure over an instantaneous threshold.

Live Demonstration Requirement

Create a controlled temperature excursion; show classification, alarm/history, and preservation/recovery of compliance state after network interruption. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 08: Smart Water Tank, Consumption Monitoring and Automatic Pump Control

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Develop a closed-loop water-management prototype that measures level and flow, controls a pump/valve, estimates consumption, and detects abnormal hydraulic operation. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class controller; tank-level and flow sensors are mandatory; pressure or pump-current sensing may be added.
- Low-voltage pump/valve through an appropriate driver/relay/MOSFET with inductive-transient protection.
- Prevent overflow and define safe behavior for failed or implausible sensor readings.

Mandatory IoT, Backend, and Application Requirements

- Compute instantaneous flow and accumulated volume; publish telemetry, actuator state, faults, and availability through MQTT.
- Provide time-series storage, REST API, and dashboard for level, flow, daily volume, pump state, alarms, and freshness.
- Automatic control must use high/low limits with hysteresis or state-machine logic; add at least one fault rule such as pump ON with near-zero flow for a defined duration.
- Manual mode must not bypass critical safety constraints; commands require acknowledgement/confirmed state.
- Essential local level control must remain operational during network outage.

Required Advanced Component

Add leak/anomalous-consumption detection using a rolling baseline, or use pump-current/flow consistency to classify pump faults. **Experimental Evaluation — Mandatory**

- Calibrate level and flow and report error against physical reference measurements.
- Measure level-control response, switching behavior, fault-detection delay, and volume-estimation error.
- Test a sensor/no-flow fault plus network loss/recovery and quantify telemetry/control latency.

Live Demonstration Requirement

Live filling/draining must update the dashboard; demonstrate automatic level control, at least one detected fault, and safe operation during network loss. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.

Project 09: Occupancy-Aware Smart Classroom for Comfort and Energy Optimization

Difficulty 5–6/10 | Team 4–5 students | End-to-End IoT System

Project Objective

Integrate occupancy and environmental sensing with automatic lighting/ventilation control to reduce unnecessary energy use while maintaining defined classroom comfort conditions. **Mandatory Hardware and Physical-System Requirements**

- ESP32-class node(s); occupancy sensing plus at least two of illuminance, temperature/humidity, and CO₂/air quality.
- At least two safe low-voltage controllable outputs representing lighting and ventilation/HVAC.
- For multiple zones, explicitly design node/zone identity and state synchronization.

Mandatory IoT, Backend, and Application Requirements

- MQTT telemetry/control architecture with documented topics, QoS, timestamps, identity, and LWT/availability.
- Backend database and REST API; real-time dashboard for occupancy, comfort variables, actuator states, alarms, energy estimate, history, and freshness.
- Control policy must use occupancy plus at least one environmental input and include hysteresis/minimum switching time.
- Provide Auto/Manual mode while preserving defined operational constraints and confirmed actuator state.
- Estimate energy from measured/characterized load power and actuator operating time.

Required Advanced Component

Implement a weighted comfort/energy score or multi-zone policy and compare it with a simpler occupancy-only controller.

Experimental Evaluation — Mandatory

- Compare Baseline versus IoT-aware control under a reproducible occupancy/environment scenario.
- Report energy saving $100(E_{baseline} - E_{IoT})/E_{baseline}$ together with comfort-violation time, so energy cannot be saved simply by disabling everything.
- Measure occupancy accuracy, end-to-end latency, switching frequency, and network-loss/recovery behavior.

Live Demonstration Requirement

Simulated occupancy/environment changes must cause real automatic lighting/ventilation actions; show history, energy comparison, manual override, and network recovery. **Common Mandatory Deliverables**

- **Working integrated prototype and source code:** embedded firmware, backend/data services, and dashboard/application code must be organized and reproducible. Sensor-to-dashboard only is incomplete.
- **Report:** Introduction/Problem; Requirements and Theory; Architecture; Hardware; Protocol/Networking; Backend/Data; Application/UI; Control/Intelligence; Security/Reliability; Experimental Method; Results/Discussion; Limitations; Conclusion; References; and member contributions.
- **60-minute presentation including Q&A:** recommended 35–40 min technical presentation, 10–15 min live demo, remaining time for questions. Every member must present and answer technical questions.
- **Technical evidence:** architecture and wiring diagrams, MQTT/API specification, database schema, source repository, screenshots, experiment tables/plots, and backup demo video.

Minimum Quality Bar

Assessment emphasizes end-to-end correctness, embedded/hardware quality, communication/backend/data design, application quality, control/intelligence, reliability/security, experimental rigor, report/presentation quality, and individual technical understanding. Features that are not demonstrated or evaluated do not receive full technical credit.